CSE 4106 - Computer Networks Laboratory

Multi-Layer Network Protocol Simulation with ARP, UDP-DNS, and ICMP

Submitted By

Ahmed Nur E Safa
Roll: 2007114

Submitted To

Md. Sakhawat Hossain
Lecturer, Department of CSE

Farhan Sadaf
Lecturer, Department of CSE

Khulna University of Engineering & Technology

Department of Computer Science and Engineering
Khulna University of Engineering & Technology

October 30, 2025

Chapter 1 – Objective

The primary objectives of this project are:

- Design and implement a multi-protocol network simulation.
- To demonstrate the interdependencies between protocols operating at different OSI layers.
- Showcase Address Resolution Protocol (ARP) for MAC address discovery.
- Implement DNS query functionality using UDP as the transport mechanism.
- Incorporate ICMP echo reply (ping) functionality for network diagnostics and connectivity testing.

Chapter 2 – Used Protocols

- **ARP (Address Resolution Protocol)**
  - **Purpose:** Translates IP addresses to MAC addresses
- **UDP (User Datagram Protocol)**
  - **Purpose:** Provides connectionless transport layer communication
- **DNS (Domain Name System)**
  - **Purpose:** Converts domain names to IP addresses
- **ICMP (Internet Control Message Protocol)**
  - **Purpose:** Network diagnostics and error reporting (ping functionality)
- **IP (Internet Protocol) - Implicit**
  - **Purpose:** Provides logical addressing and routing

Chapter 3 – Introduction

Modern computer networks rely on a hierarchical stack of protocols, each serving specific functions at different layers of the OSI model.

Address Resolution Protocol (ARP)

The Address Resolution Protocol (ARP) operates at the boundary between the Network Layer (Layer 3) and the Data Link Layer (Layer 2) of the OSI model. Its primary function is to resolve IP addresses to physical MAC addresses. ARP maps 32-bit IP addresses to 48-bit MAC addresses and facilitates communication by enabling devices to locate each other on the same network segment. The protocol operates through a process that begins when a device needs to send data to another IP address on the local network. First, it checks its ARP cache to see if it already has the corresponding MAC address. If the mapping is not found, the device broadcasts an ARP request to all devices on the local network, asking, in effect, "Who has this IP address?" The device that owns the requested IP address responds directly with a unicast reply containing its MAC address. Upon receiving this reply, the requesting device stores the IP-to-MAC mapping in its ARP cache for future use.

Domain Name System (DNS)

The Domain Name System (DNS) is a hierarchical and distributed naming system designed to translate human-readable domain names such as `https://aiexaminer.onrender.com` into machine-readable IP addresses. DNS typically relies on the User Datagram Protocol (UDP) on port 53 for queries and responses. When a client needs to resolve a domain name, it sends a DNS query packet containing the requested domain name to a DNS server. The server then searches its local records or consults other servers in the DNS hierarchy to locate the corresponding IP address.

Internet Control Message Protocol (ICMP)

The Internet Control Message Protocol (ICMP) is a supporting protocol within the Internet Protocol (IP) suite, used for diagnostic and control functions at the Network Layer (Layer 3) of the OSI model. It conveys error messages and operational information that help manage and troubleshoot IP networks. One of its most well-known applications is the ping utility, which leverages ICMP's echo request and echo reply messages to test connectivity between two hosts. During a ping operation, the source device sends an ICMP echo request packet that includes a sequence number to uniquely identify the message. This mechanism confirms reachability, providing valuable insights into network latency.

User Datagram Protocol(UDP)

UDP is a lightweight, connectionless transport layer protocol used for fast and efficient data transmission across IP networks. Unlike TCP, UDP does not establish a connection before sending data, nor does it guarantee delivery, order, or error correction, making it ideal for time-sensitive applications where speed matters more than reliability. Commonly used in DNS queries, streaming media, online gaming, and VoIP, UDP encapsulates application data into datagrams with minimal overhead, adding only a small header containing source and destination port numbers, length, and an optional checksum. In this simulation, UDP serves as the transport mechanism for DNS messages, using port 53 as the standard destination port,

thereby demonstrating how higher-layer protocols rely on transport services to communicate across networked devices while maintaining simplicity and low latency.

3.1 The OSI Model Context

The OSI (Open Systems Interconnection) model is a conceptual framework used to understand and standardize how different networking protocols and technologies interact across a network.The simulation demonstrates protocol interaction across multiple OSI layers:

- **Layer 2 (Data Link):** ARP for MAC address resolution
- **Layer 3 (Network):** IP addressing, ICMP for diagnostics
- **Layer 4 (Transport):** UDP for connectionless data transport
- **Layer 7 (Application):** DNS for name resolution, HTTP concepts for web content

Chapter 4 – Methodology

The project is implemented using OMNeT++ (Objective Modular Network Testbed in C++), a powerful discrete event simulation framework designed specifically for network simulations. The simulation consists of 4 main components connected through a central router:

- **PC (Client):** Initiates network requests and performs DNS lookups
- **DNS Server:** Resolves domain names to IP addresses
- **HTTP Server:** Represents the web server hosting content
- **Router:** Intelligently forwards packets between network segments

4.1 Message Definitions

Custom message types were defined using OMNeT++'s message definition language:

1. **ARPPacket:** Contains source or destination IP and MAC addresses, request or reply flag
2. **UDPPacket:** Contains source or destination ports for transport layer addressing
3. **DNSPacket:** Contains query name, answer IP, and query flag
4. **ICMPPacket:** Contains source or destination IPs, sequence number, and request flag

4.2 Module Implementation

The simulation's modular design faithfully replicates real-world network behavior through four key components.

PC Module

The PC module acts as an intelligent client that initiates network activity: it maintains a local ARP table to cache resolved MAC addresses, schedules periodic ICMP ping requests for connectivity monitoring, and triggers DNS queries until domain resolution succeeds. It carefully tracks DNS resolution state and handles incoming ARP replies, ICMP echoes, and DNS responses, while ensuring all DNS traffic is properly encapsulated in UDP packets demonstrating layered protocol interaction in action.

DNS Server Module implements

The DNS Server module functions as a dedicated name resolution service. Beyond responding to ARP requests for its own IP, it listens for UDP packets on port 53, the standard DNS port. Upon receiving a query, it parses the requested domain, performs a simple lookup (mapping example.com to 192.168.0.20), and crafts a DNS response, which it wraps in a UDP reply with correctly swapped source and destination ports ensuring the client can match the response to its original request.

HTTP Server Module implements

The HTTP Server module, while not implementing full HTTP logic in this phase, provides essential network-level services: it replies to ARP requests for its IP address and responds to ICMP echo requests, enabling basic reachability testing. This lays a solid foundation for future extension into actual web content delivery, aligning with the project's goal of simulating end-to-end web browsing.

Router Module implements

The Router module serves as the intelligent traffic director. It implements protocol-aware forwarding: ARP requests are broadcast to all subnets to ensure resolution across segments, while UDP packets destined for port 53 are explicitly routed to the DNS server. All other traffic (including ICMP and future HTTP) is forwarded to the HTTP server. This port and protocol-based routing logic mimics real router behavior, enabling clean separation of services and demonstrating how network infrastructure enables multi-protocol communication.

4.2.1 ARP Table Implementation

To support dynamic MAC address learning, a dedicated ARPTable class was implemented. This lightweight yet efficient structure stores IP-to-MAC address mappings, allowing modules to add entries, retrieve MAC addresses by IP, and check for existing mappings. By persisting learned addresses throughout the simulation, it avoids redundant ARP requests and mirrors the behavior of real network interfaces, significantly enhancing the realism and efficiency of Layer

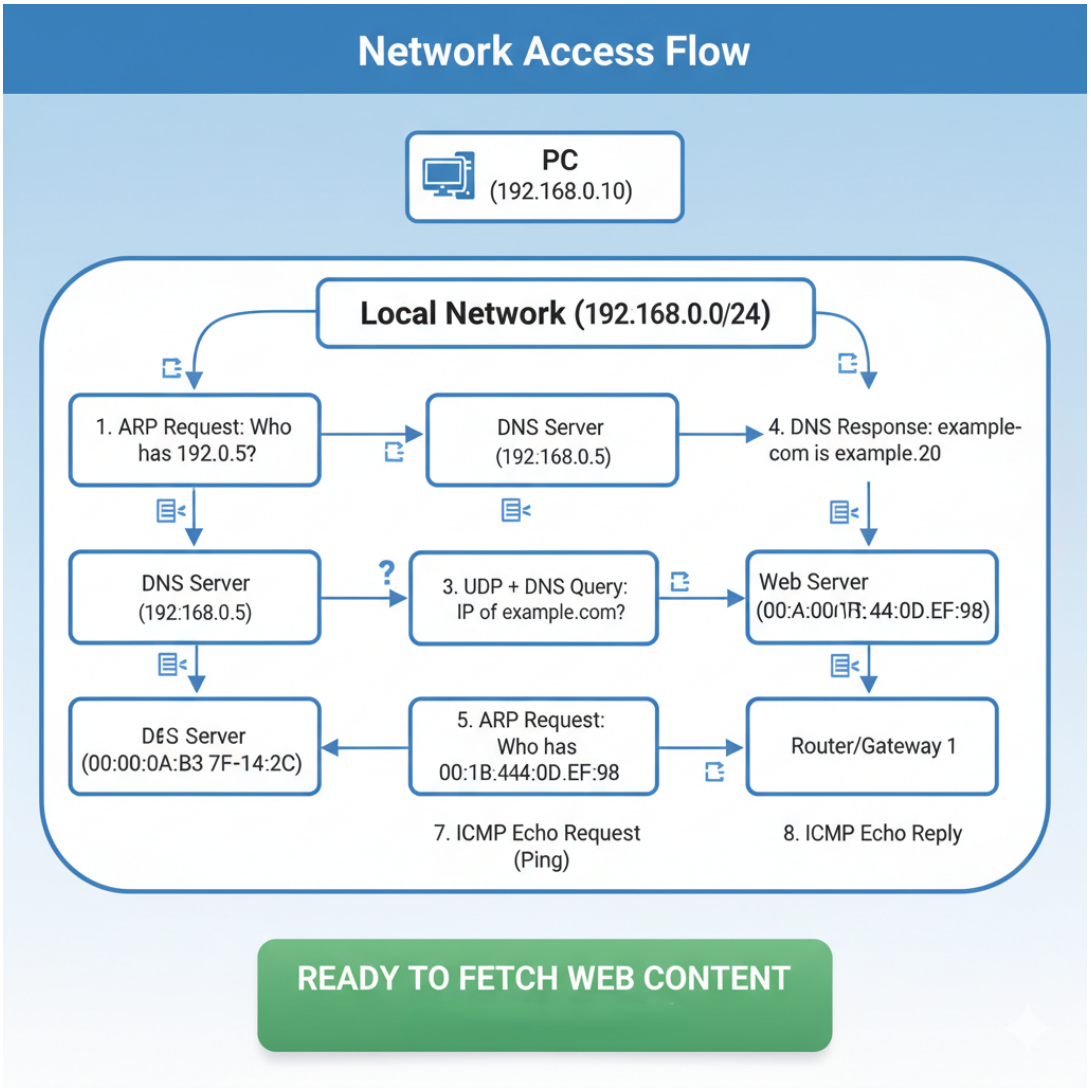

Figure 5.1: Workflow

2 communication in the simulated environment.

Chapter 5 – Working of the Simulation

5.1 Initialization Phase

When the simulation starts at time t=0:

- All modules initialize with configured IP and MAC addresses
- PC schedules ping timer at t=0.5s
- PC schedules DNS query timer at t=1.0s
- All modules are ready to process incoming messages

5.2 First Ping Attempt

1. PC's ping timer triggers, attempting to ping DNS server (192.168.0.10)
2. PC checks ARP table for DNS server's MAC address not found
3. PC sends ARP request:
   - Source IP: 192.168.0.1
   - Source MAC: AA:BB:CC:DD:EE:01
   - Destination IP: 192.168.0.10
   - Destination MAC: FF:FF:FF:FF:FF:FF
   - Request Flag: true
4. Router receives ARP request on in PC port and broadcasts to all servers
5. DNS Server receives ARP request, recognizes its IP address, and replies:
   - Source IP: 192.168.0.10
   - Source MAC: AA:BB:CC:DD:EE:10
   - Destination IP: 192.168.0.1
   - Destination MAC: AA:BB:CC:DD:EE:01
   - Request Flag: false
6. Router forwards ARP reply back to PC
7. PC receives ARP reply and updates its ARP table:
   - IP: 192.168.0.10 $\rightarrow$ MAC: AA:BB:CC:DD:EE:10

5.3 DNS Resolution

1. PC's DNS timer triggers, initiating domain resolution
2. PC checks if DNS server's MAC is known yes (learned from previous ARP)
3. PC creates DNS query packet:
   - Query Name: "example.com"
   - Is Query: true
4. PC encapsulates DNS in UDP packet:
   - Source Port: 10053 (client DNS port)
   - Destination Port: 53 (DNS server port)
   - Payload: DNS Query
5. Router receives UDP packet, examines destination port 53, routes to DNS server

6. DNS Server receives UDP packet on port 53, extracts DNS query
7. DNS Server performs lookup:
   - Query: ”example.com” → Answer: ”192.168.0.20”
8. DNS Server creates response:
   - Query Name: ”example.com”
   - Answer IP: ”192.168.0.20”
   - Is Query: false
9. DNS Server encapsulates response in UDP:
   - Source Port: 53
   - Destination Port: 10053 (reply to client port)
   - Payload: DNS Response
10. Router forwards UDP packet to PC
11. PC receives UDP packet on port 10053, extracts DNS response
12. PC updates internal state:
    - resolvedIP = ”192.168.0.20”
    - dnsResolved = true
13. PC logs: ”DNS resolved example.com → 192.168.0.20”
14. PC logs: ”Fetching web content from 192.168.0.20”

Chapter 6 – Real-World Applications

The simulation effectively models several critical real-world networking scenarios, demonstrating how foundational protocols interact in practical environments.

6.1 Corporate Network Infrastructure

In corporate network infrastructure, the behavior closely mirrors that of enterprise settings. Client machines routinely perform DNS lookups to access internal services (like file servers) or external websites, while continuous ICMP ping requests emulate network monitoring tools that ensure server availability and performance. The router's ability to forward traffic based on protocol type and port number reflects real enterprise routing policies that segment DNS, web, and management traffic for security and efficiency. Meanwhile, the dynamic maintenance of ARP tables replicates how devices on the same LAN segment resolve and cache MAC addresses to enable fast, direct Layer 2 communication essential for day-to-day operations in office networks.

6.2 Internet Service Provider Networks

Similarly, the architecture aligns with core practices in Internet Service Provider (ISP) networks, where scalability and reliability are paramount. ISPs deploy high-performance DNS servers that handle millions of resolution requests daily, forming the backbone of internet usability. Their routers implement sophisticated forwarding and filtering rules much like the simulation's port-based logic to manage traffic flow, enforce policies, and prevent congestion. ICMP remains indispensable for ISPs in diagnosing path failures, measuring latency, and troubleshooting connectivity issues across vast networks. Crucially, the seamless coordination between ARP (Layer 2), IP/ICMP (Layer 3), UDP (Layer 4), and DNS (Layer 7) underscores how multi-layer protocol harmony enables the global internet to function reliably at scale.

6.3 Modern Data Center Operations

Finally, the simulation captures key dynamics of modern data center operations, where automation and low-latency communication are critical. Health-check mechanisms, such as periodic pings, are widely used by load balancers and orchestration systems to detect server outages and reroute traffic in real time exactly as modeled by the PC's ping timer. DNS-based service discovery allows microservices and applications to locate backend resources dynamically, a pattern reflected in the PC's resolution of example.com before initiating communication. Within data center fabrics, ARP operates inside VLANs or overlay networks to resolve endpoints within the same broadcast domain, while UDP is often preferred over TCP for time-sensitive applications like DNS, monitoring telemetry, and real-time analytics due to its minimal overhead and faster transmission—highlighting why UDP was deliberately chosen for DNS transport in this project. Together, these elements showcase how even a simplified simulation can reflect the complex, interdependent protocol ecosystem powering today's digital infrastructure.

Chapter 7 – Discussion

This multi-layer network protocol simulation demonstrates the intricate cooperation required among different networking protocols to accomplish even simple tasks like resolving a domain name and testing connectivity. The choice to implement DNS over UDP rather than as standalone messages reflects actual internet architecture, where UDP provides the necessary transport layer framing for port-based routing while maintaining the low overhead and quick response times that make DNS efficient for billions of daily queries. Similarly, the decision to have each module maintain its own ARP table accurately models how real network stacks operate, where devices independently manage their MAC address caches, leading to realistic behavior including the initial latency of address resolution and the need for periodic ARP requests.

The router's intelligent forwarding mechanism represents a simplified but functionally accurate model of real routing decisions, using protocol type identification through dynamic casting and port-based inspection to direct traffic appropriately. The simulation clearly demonstrates the fundamental dependency chain in network protocols. Higher-layer protocols cannot function without lower-layer services, as evidenced by DNS queries requiring prior ARP resolution, and application-layer operations depending on successful name resolution.

The simulation successfully captures the essential behavior patterns and protocol interac-

tions that occur in real networks. The observable protocol dependency, state management requirements, and timing coordination challenges provide valuable insights into why network operations sometimes fail and how proper sequencing and caching optimize performance. The implementation challenges encountered in string handling in OMNeT++, message encapsulation and ownership management, timing coordination between dependent operations, and type-based routing logic represent real engineering problems that network software developers face daily.

Chapter 8 – Conclusion

This multi-layer network protocol simulation successfully achieves its primary objective of demonstrating how fundamental networking protocols cooperate across different OSI layers to enable modern network communication. Through the simulation of a complete web browsing workflow from initial ARP resolution through DNS query processing to ICMP connectivity testing, the project illustrates the intricate dependencies and careful coordination required for even routine network operations.

Each protocol fulfills its specific role: ARP bridges the gap between logical IP addressing and physical MAC addressing at Layer 2, UDP provides efficient connectionless transport at Layer 4, DNS translates human-readable domain names into routable IP addresses at Layer 7, and ICMP offers essential diagnostic capabilities at Layer 3.

The simulation reveals that seemingly simple operations like visiting a website actually require multiple protocol interactions executed in precise sequence, with each step depending on the successful completion of prerequisite operations. The project ultimately demonstrates that network complexity emerges from relatively simple protocol rules.

Chapter 9 – References

This simulation was developed based on official protocol specifications and the OMNeT++ simulation framework. The following resources were essential for implementation.

Protocol Specifications

1. **ARP RFC 826** – `https://datatracker.ietf.org/doc/html/rfc826`
2. **UDP RFC 768** – `https://datatracker.ietf.org/doc/html/rfc768`
3. **ICMP RFC 792** – `https://datatracker.ietf.org/doc/html/rfc792`
4. **DNS RFC 1035** – `https://datatracker.ietf.org/doc/html/rfc1035`

Simulation Tools

5. **OMNeT++ Official Site** – `https://omnetpp.org/`

Learning Resources

7. **Network Protocols Guide** – https://www.geeksforgeeks.org/computer-network-tutorials/
8. **OMNeT++ Community** – https://omnetpp.org/community/
9. **Network Lessons** – https://networklessons.com/
10. **OMNeT++ Tutorial Videos** – https://www.youtube.com/c/OMNeTpp
11. **INET Framework Examples** – https://github.com/inet-framework/inet/tree/master/showcases
12. **Wireshark Protocol Analysis** – https://www.wireshark.org/docs/
13. **TCP/IP Guide** – http://www.tcpipguide.com/
14. **Study-CCNA Networking** – https://study-ccna.com/
15. **OMNeT++ Stack Overflow** – https://stackoverflow.com/questions/tagged/omnet++